SNN + RRAM STDP Simulation Guide

Sheng Lu

May 3, 2026

1 Overview

This project implements a hardware-level **Spiking Neural Network (SNN)** simulation using:

- HSPICE for circuit simulation
- RRAM devices as synaptic weights
- Spike-Timing-Dependent Plasticity (STDP)
- MATLAB for automation and visualization

2 Simulation Workflow

2.1 Step 1: Generate SPICE Netlist

```
generate_spice_model_inhabi('stdp.sp', rootpath, N, M, base_gap,
   gap_sigma, timestep, simulationtime)
```

2.2 Step 2: Run HSPICE

```
system(cmd);
```

2.3 Step 3: Read Results

```
[res, ~] = import_data('out.lis');
```

2.4 Step 4: Visualization

- RRAM weight map
- Conductance evolution
- Membrane voltage

3 System Architecture

- M neurons
- Each neuron has N input channels
- Each input is connected through one RRAM synapse

Key features:

- STDP learning
- Spike generation
- Lateral inhibition (winner-take-all behavior)

4 Controllable Parameters

4.1 Network Size

```
M = 4;   % number of neurons
N = 64;  % inputs per neuron
```

Impact:

- Determines network scale
- Affects simulation time and memory usage

4.2 Time Parameters

```
timestep = 0.2;         % us
simulationtime = 4000; % us
```

Parameter	Effect
timestep	Time resolution (smaller = more accurate, slower)
simulationtime	Total simulation duration

Recommendation:

- timestep: $0.1 \sim 1$ μs

4.3 RRAM Initial Gap (Critical)

```
base_gap = 1.0e-9;
gap_sigma = 0.02;
```

```
gap_value = base_gap * (1 + gap_sigma * randn());
```

Parameter	Description
base_gap	Initial filament gap
gap_sigma	Gaussian variation (device mismatch)

Impact:

- Determines initial resistance distribution
- Strongly affects learning dynamics

4.4 RRAM Dynamic Parameters

```
Xrram ... gap_ini=gap_ini_base tstep=...
```

- Controls STDP update speed
- Affects convergence stability

4.5 Spike Generator Parameters

```
spikegen vth=0.8 a=-0.20 b=0.0 Tr=3us
```

Parameter	Meaning
vth	Threshold voltage
a, b	Pulse shaping parameters
Tr	Pulse width

4.6 Neuron Circuit Parameters

```
C1 node1 node2 1n
R1, R2, R3, R4, R5
```

- C1: membrane capacitance (integration)
- Resistors: control gain and time constant

4.7 Lateral Inhibition

```
M_m_k node2_m vout_SRC_k ...
```

Function:

- Enables neuron competition
- Implements winner-take-all behavior

Effect:

- Too strong: only one neuron fires
- Too weak: multiple neurons fire

4.8 Input Data

```
.include 'mnist_digit1_20samples_7.sp'
```

- Defines input spike patterns
- Can be replaced with custom datasets

5 Output Data Interpretation

5.1 Weight Map

```
weights = reshape(1 ./ res(end, rram_cols), 8, 8);
```

- Shows final synaptic weight distribution

5.2 Conductance Evolution

```
plot(time, 1 ./ R)
```

- Shows STDP learning over time

5.3 Membrane Voltage

```
plot(time, Vmem)
```

- Indicates neuron firing behavior

6 Parameter Tuning Guidelines

6.1 If no convergence

- Reduce gap_sigma
- Increase simulationtime
- Adjust spike parameters

6.2 If all neurons fire

- Increase lateral inhibition
- Raise threshold

6.3 If no learning

- Check spike timing
- Adjust RRAM parameters
- Reduce timestep

6.4 If simulation is too slow

- Reduce N
- Reduce simulationtime
- Increase timestep (carefully)

7 Recommended Default Configuration

```
M = 4;
N = 64;

timestep = 0.2;
simulationtime = 4000;

base_gap = 1e-9;
gap_sigma = 0.02;
```

8 Conclusion

This framework provides a complete hardware-level SNN simulation platform combining:

- RRAM-based synapses
- STDP learning
- Neuron dynamics
- Lateral inhibition

Core tuning focuses on:

- RRAM initial distribution (gap)
- Spike timing
- Neuron competition